資料結構與攤銷分析：獨立練習

A04-E01 搬移只能在 T 空時進行

從空 queue 開始，依序 ENQ a、ENQ b、DEQ、ENQ c、DEQ、DEQ。兩個 stack 均以底→頂列出。填每步的 S, T、出列結果、primitive 成本與 $\Phi = 2|S|$。若在 T 非空時也把 S 倒進去，哪一次出列會錯？

A04-E02 勢能前面的係數要多大

令 $\Phi_a = a|S|$，其中常數 $a \geq 0$。推導 ENQUEUE、普通 DEQUEUE、搬移 k 筆後 DEQUEUE 的攤銷成本。哪些 a 能使這三類操作都有與 k 無關的常數上界？

A04-E03 非空初態不能免費

初態 $S = [a, b, c, d]$（底→頂），T 空，只做一次 DEQUEUE。以 $\Phi = 2|S|$ 計算 $c, \Delta\Phi, \hat{c}$，並用望遠鏡等式核對。為何不能宣稱這一次實際只需 1 單位？

A04-E04 擴張與縮表的逐步總帳

採正文 APPEND／DELETE-LAST 規格，從 $(0, 0)$ 依序加入 a,b,c,d,e，再刪除 e,d,c,b,a。列每步 (n, M)、搬移數、$c, \Phi, \hat{c}$，核對總帳。

A04-E05 半滿就縮表的反例

錯誤策略為滿表倍增、刪到半滿立即減半。先從空表連續 APPEND 8 筆，再重複 4 次「APPEND x、DELETE-LAST」。列狀態循環與精確成本；與 $3m$ 比較，並推廣為非線性總成本的操作族。

A04-E06 勢能加常數是否破壞證明

原勢能 $\Phi \geq 0$ 且空初態 $\Phi_0 = 0$。改成 $\Psi = \Phi + 7$。有人說「$\Psi_0 \neq 0$，所以勢能法失效」。判斷並推導正確總界。

A04-E07 常數攤銷不等於常數延遲

兩個 stack queue 與動態陣列均有常數攤銷界。各給一個單次線性成本的可達狀態，並說明若系統要求每次呼叫都有固定延遲上限，還缺少什麼保證。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A04-E08 中間刪除的成本不在原定理裡

在連續陣列中保持元素順序，新增操作 DELETE-FIRST，實作為搬移全部剩餘元素。先 APPEND n 筆，再 DELETE-FIRST n 次。即使完全不縮表，總成本是否仍為 O(n)？列出搬移級數並指出規格差異。

A04-E09 四位元溢位的帳目

固定四位元、模 16 加一，初態 1101。做三次 INCREMENT，列每次 bit 字串、實際翻轉數、勢能及攤銷成本。若有人宣稱這三次實付至多 2×3，哪個前提被漏掉？

A04-E10 MULTIPOP 與 MULTIPUSH 的計費單位

從空 stack 依序 PUSH a、PUSH b、PUSH c、MULTIPOP(2)、MULTIPOP(9)。僅計成功 push／pop 成本，列每次實付與餘額，採 PUSH 收 2、其他收 0。再加入單一呼叫 MULTIPUSH(k) 寫入 k 個元素，能否仍對每個 API 呼叫收常數？

A04-E11 加一與減一交替的具體反例

四位元計數器從 0 開始，先加一 7 次，再重複四組 INCREMENT、DECREMENT。列準備成本、交替兩個狀態、每組成本及全程成本。再把反例推廣至 k 位元，保留建構初態所需的呼叫，證明常數攤銷界失效。

A04-E12 滿佇列拒絕插入後還能正確出列嗎

容量 C=3，從空態依序 ENQ A、B、C、D，DEQ，ENQ E，再 DEQ 三次。列每步 h、s、實體陣列及回傳；失敗插入 D 是否可以先推進 h？

A04-E13 反轉時為何要先存 nxt

串列 a→b→c→NIL，依正文反轉。列每輪結束時 prev 路徑、cur 路徑，說明不變量。若先 cur.next=prev，再 cur=cur.next，第一輪後會怎樣？

A04-E14 只有一筆資料的 sentinel 串列

空態 z.next=z.prev=z。先把 x 插在 z 後面，再刪 x。列四次插入寫入與兩次刪除寫入的目標和值；指出為何刪除 z 不合法。

A04-E15 回收槽位之後，舊 handle 還代表原物件嗎

採正文五槽物件池。配置槽 1 並把 D 接在 C 之後，再刪槽 2 的 B 並回收。列 head、free 與兩條鏈。接著再配置一次，取得哪個槽？若舊 handle=2 被當成 B 使用，問題是甚麼？

A04-E16 繞回、刪除與同鍵更新

m=5、線性探測、home(k)=k mod5。依序 INSERT(4,a)、INSERT(9,b)、INSERT(14,c)、DELETE(9)、INSERT(14,c′)、INSERT(19,d)、FIND(24)。列每次探測槽、最後表內容與活躍筆數。為何不能在 DELETED 立即放入 14？

A04-E17 Affine family：真的枚舉一次

P=5、m=3，a∈{1,2,3,4}、b∈{0,1,2,3,4} 均勻抽樣。對鍵 x=1、y=4，逐個 a 列出造成碰撞的 b，計算碰撞機率。若結果不是 1/3，是否違反 universal？

A04-E18 步長不互質的代價

m=8、home=1，分別列 step=2 與 step=3 的探測順序及週期。如果 step=2 可到的槽全滿，但槽 0 仍空，能插入嗎？再列三角偏移 i(i+1)/2 mod8 的順序，說明除以 2 的時機。

A04-E19 理想模型的精確期望與上界

m=4，固定槽 0、1 占用，槽 2、3 為 EMPTY，query 的探測排列均勻分布於 24 種排列。算 Pr(X≥1)、Pr(X≥2)、Pr(X≥3) 及 E[X]，再與 1/(1−α) 比較。為何不可把前兩次命中占用的機率寫成 α^2？

A04-E20 FKS 空間與成員資格

使用正文六鍵 FKS 配置。算平方槽數 S 與第一層碰撞對數 Y，驗證 S=n+2Y；逐步查 26、14、5、10。有人讀到非空槽就回成功，哪幾個查詢會錯？

A04-E21 刪空之後，為何不是一次就查完

線性探測表 m=5，依序插入 0～4，再全部刪除，只留下 tombstone。算 α、β 與 FIND(7) 的探測順序。這是否反證了理想 uniform probing 的失敗搜尋定理？

A04-E22 合數 modulus 破壞哪一步？

把 MIT 一維 dot-product family 改成模 4：a 均勻取 0～3，h_a(x)=ax mod4。固定 x=0、y=2，算碰撞機率，指出模質數證明哪一步失效。

A04-E23 父子關係都對，整棵樹卻錯了

root=20，左孩子 10、右孩子 30，10.right=25。逐邊比較看似合法，指出違反哪個祖先區間。若只沿既有連結執行 FIND(25)，會走哪條路、得到什麼結果？

A04-E24 相同樹，不同查詢分布

使用 root5、孩子2與8、2.right=3、8.left=7 的樹。先算均勻成功查詢的平均比較數；再令 query 在1～10均勻分布，算混合平均。若 query=9 的機率改成1，其餘為0，平均又是多少？

A04-E25 後繼的兩條路徑

仍用前題的樹。分別追蹤 SUCCESSOR(5)、SUCCESSOR(3)、SUCCESSOR(8)，寫出每次向下或向上的節點。為什麼不能把「沒有右孩子」直接當成「沒有後繼」？

A04-E26 直接後繼為什麼不能照抄深處分支

root z=20，z.left=10，z.right=y=40，y.right=50。若不檢查 y.parent!=z 就執行 y.right=z.right，會發生什麼？列出正確刪除後的 root、parent 與 child 連結。

A04-E27 後繼帶著右孩子的根刪除

插入20,10,40,30,25,27,50。以移接法刪除20，逐步列出所有改寫的child與parent。若改採複製payload法，哪些外部handle的意義或有效性不同？

A04-E28 六種排列的期望高度

對key集合{1,2,3}枚舉六種插入順序。高度按邊數算，求$E[X_3]$與$E[2^{X_3}]$。比較$2^{E[X_3]}$與$E[2^{X_3}]$，並指出五種樹形等機率的模型會得到哪個高度平均。

A04-E29 固定秩與平均秩的期望搜尋成本

五個相異key隨機順序建BST。求秩3的期望深度、成功查詢比較數，以及五個key均勻成功查詢的平均比較數期望。列出調和數與代入步驟。

A04-E30 指數高度證明的基例與不等號

有人寫「$Y_0=2^{-1}$，但同時$Y_0=0$；E[max(U,V)]=max(E[U],E[V])」，再得到高度界。指出兩處問題，寫出合法遞迴上界，並完成$a_n \le n^3$到$E[X_n] \le 3\log_2 n$的步驟。

A04-E31 • 黑高與路徑黑數

root20B，左右為10R、30R，其餘 child 都是黑 NIL。求 bh(20)、bh(10)、bh(NIL) 及含 root 的完整路徑黑數。再把兩個紅 child 改黑，重算以上數量。

A04-E32 • 左旋的外父與空子樹

p30.left=x10，x.right=y20，y.left=β15。列出左旋的六個欄位寫入，說明若 x 原為 root 要改哪兩項；若 β=NIL，哪項省略？為何不能更新「y 的原父指向 x」？

A04-E33 • 非根 I1 的黑數

g黑、p與u紅，各條p/u以下路徑（不含p/u、含末端NIL）有b個黑節點。I1把g紅、p/u黑。分別算改色前後的bh(g)與穿過g的完整路徑黑數。若g是root，最後塗黑又改變什麼？

A04-E34 • I2 不能提前返回

由空樹依序插35、15、25，列出插入後、I2後、I3後的root、各parent/child、顏色及z指向；再用鏡射順序15、35、25核對旋轉方向。

A04-E35 • 高度證明的計數邊界

以內部邊數高度h及root到外部NIL邊數H表示同一棵樹。證明子樹至少有$2^{bh(x)}-1$個內部節點，再導出$H \le 2\log_2(n+1)$。列空樹、單節點的h與H，說明O(log(n+1))為何不能直接作含空樹的操作成本上界。

A04-E36 • 不同修復時機的比較

有人說：「每層至多兩次旋轉，所以CLRS紅黑樹插入共O(log n)次旋轉；top-down插入跟它只是翻譯不同。」指出兩句話的問題，區分時間成本、基本旋轉數及演算法版本。

A04-E37 • 原色與移動後顏色

依序插40、20、60、10、30、50、70、45、55，再刪40。列z、y、x及y的原色、移動後顏色，追蹤NIL.parent；為何不能看移動後y黑就呼叫修復？

A04-E38 • 最後一點與紅 child 吸收

分別刪除單節點25B，以及root25B、唯一左child15R中的25。列迴圈是否執行、最後root與顏色；說明extra black是證明記號而不是第三種color。

A04-E39 • D1 後為何還要 D2

插15、25、35、45、55、65後刪15。追蹤缺黑位置、p、w、D1旋轉及D2回傳值。列最終每個節點顏色，指出哪個紅節點吸收extra。

A04-E40 • D3→D4 的逐路黑數

插15、25、45、35後刪15。從25B→NIL(x)及25B→45B→35R→NIL兩條路出發，分別列D3前、D3後、D4後的實際黑數、extra與有效黑數，並列最終樹。

A04-E41 • 右側缺黑與直接後繼

插15、35、45、25後刪35。後繼是誰？x=NIL時parent應指哪個物件？以near／far角色列出鏡射D3、D4與最終樹。

A04-E42 • 為何刪除最多三次旋轉

證明只有D2能讓修復再往上一層，且D1、D3、D4在整次刪除中各至多一次。說明「最多三次旋轉」為何不代表刪除O(1)。

A04-E43 • 局部秩與全域秩

使用正文十鍵樹，逐層追蹤 select(7)，再從節點 27 算 rank。列出每次 k、r 與增加的前綴，並說明空樹和重複鍵的契約。

A04-E44 • 後繼搬移後的摘要

在同一十鍵樹刪 20，後繼為 25，且 25.right=27。列出重算 size 的起點、順序與數值；說明為何從舊根的 parent 開始不對。

A04-E45 • 局部公式不等於只改兩格

為何 size 與 max 在旋轉後只需局部更新，height 卻可能影響祖先？寫出 left rotation 的更新順序，並分析把每個節點的全域 rank 直接存下來的代價。

A04-E46 • max 大一點也不安全

將正文左子樹的 [8,15] 改為 [8,27] 後刪掉它。假設 [5,7].max 錯留 27。追蹤 query=[22,22]，指出被漏掉的答案，並用一般剪枝證明解釋為何上界不夠。

A04-E47 • 相接端點與欄位更新

比較 [4,5]、[5,7] 與半開版本。列兩種 overlap 與 left.max 判斷；若要改 low 或只改 high，各須做什麼？

A04-E48 • 列全部答案的成本

重複查詢、暫刪命中區間，最後插回。若共有 k 個答案，為何完整上界含 k+1 而非只有 k？是否可直接宣稱 O(log n+k)？

A04-E49 • 兩種容量參數與分裂時機

order m 表示最大分支數。分別寫出 m=3、m=4 與 t=2 的非根鍵數範圍。為何不能對 m=3 的滿葉 [2,8] 使用「插入前先分裂」？

A04-E50 • 十億鍵的高度與頁讀取

t=100、n=10^9。利用稀疏與密集兩界求可能高度。根常駐記憶體，其餘路徑頁未快取時，搜尋最深葉需幾次頁讀取？

A04-E51 • 內節點分裂的全部子指標

t=2，父 [50]，左內節點 [10,20,30] 連葉 [5]、[15]、[25]、[35]；右內節點 [70] 連 [60]、[80]。分裂滿左子節點，列所有父鍵與子指標，核對每筆鍵恰出現一次。

A04-E52 • 借鍵再合併的連續刪除

使用正文刪5的樹。逐步說明父20、兄弟30與子樹25的移動，再列合併[5]與[15]後的最終樹。只搬鍵不搬[25]會破壞哪個範圍？

A04-E53 • 前驅與後繼的資料一致性

兩棵 t=2 樹均為根[20]。A的葉為[5,10]、[30]；B為[5]、[30,40]。分別刪20，列替換鍵與最終樹；若 record20 的資料留在根會怎樣？

A04-E54 • 失敗刪除也要處理根

t=2，根[20]、葉[5]與[30]，刪99。列回傳值、合併索引與最終樹。刪除包裝函式若在 NOT_FOUND 時立刻退出會留下什麼問題？

A04-E55 • 根為何可以縮高

根[20]、葉[5]與[30]，刪20，再依序刪5、30。列每步的根及高度，並證明不能在任意內節點用唯一child取代它。

A04-E56 • 頁內二分搜尋能改善哪些成本

說明搜尋、插入與刪除的頁存取與CPU成本。為何把lower_bound換成二分搜尋，不代表所有操作都只需每層O(log t)？

A04-E57 • 環串列與合併契約

兩個非空環的入口為 a、b。列出接環所需指標更新，並說明為何不可以把兩個仍可獨立操作的 heap 當成 UNION 的結果與輸入同時保留。

A04-E58 • 完整整理與孩子提升

依正文順序，插入 1,4,9,7,12,6,15,10 後取出 1，列全部 link 與終態。再取出 4，固定先處理根 10、6，再處理提升的孩子 9、7，列新森林與 min。

A04-E59 • 降低鍵值不等於單獨搬走一個值

根1→12→15→20為一條父子鏈，12未標記。分別從此初態將15改為14及11，列根、parent、degree、mark與min變化。

A04-E60 • 祖父為根與非根的兩個結局

使用正文兩個級聯例。把x=13降到1，p=8原本已標記。分別列兩次CUT後的根、mark與(r,m)，不可只回答「g被標記」。

A04-E61 • 攤銷證明中的常數不能憑空抵銷

若級聯共切c個節點，實際成本≤a(c+1)，證明同一Φ=K(r+2m)如何給出常數攤銷成本。為何寫O(c)−c=O(1)本身還不夠？

A04-E62 • 不用樹高證明degree界

解釋按接入時間排列的第i個孩子為何目前degree至少i−2。以子樹大小強歸納證明size(x)≥F_{d+2}，並推出最大degree界。

A04-E63 • 重複最小值與刪除身分

兩個不同節點u、x都有一般鍵−∞。若DELETE(x)只再次設定x.key=−∞，何時會刪錯？給出能確保刪x的比較規則及攤銷成本。

A04-E64 • 延遲工作的真實代價

從空堆積插入n個互異鍵再取出最小值。解釋該次取出為何可花Θ(n)，卻不違反O(log(n+1))攤銷界。若從任意既有森林開始，序列總成本還須加什麼？

A04-E65 • 奇數位元寬度與局部座標

u=32 時，分解 21、30；再把 21 的 low 傳入對應 cluster，繼續分解。列出每一層的 u、H、L、high、low，並檢查還原。

A04-E66 • 重建集合與三種後繼

使用 S={2,3,7,18,21,30}、u=32。列出頂層 min/max、所有非空 cluster 的局部集合與 summary。逐步求 SUCCESSOR(3)、SUCCESSOR(20)、SUCCESSOR(30)。

A04-E67 • 前驅不是後繼的完全鏡像

同一集合中求 PREDECESSOR(3)、PREDECESSOR(2)、PREDECESSOR(31)。若summary前驅不存在就回NIL，哪一題會錯？為何？

A04-E68 • 換出舊 min 與首次非空的組

先由原集合插入1，再插入10。各步列出頂層min/max、改動的cluster與summary，說明哪一個步驟可能遞迴更新summary。

A04-E69 • 提升最小值與縮成單一元素

分別由 S={2,3,7,18,21,30} 與 T={2,3} 刪除2。列出新min、刪除的局部鍵、summary變化、最後max，並說明程式變數x何時改值。

A04-E70 • 兩次遞迴呼叫的真正成本

一般刪除程式先刪cluster，可能再刪summary。證明兩者不會同時產生非定值遞迴成本；再由位元寬度推導最壞時間，保留奇數寬度與u=2基例。

A04-E71 • 完整配置的空間歸納

令 u=HL，$H = 2^{\lceil m/2 \rceil}$、$L = 2^{\lfloor m/2 \rfloor}$，且 $P(u) \le HP(L) + P(H) + cH$。用 $P(u) \le A(u-1) - B$ 證明線性上界，給出A、B所需條件，並解釋為何初始化時間不能忽略。

A04-E72 • 違反前提會如何破壞資料

u=4、集合只含{1}。直接呼叫DELETE-PRESENT(2)會發生什麼？集合只含{1}時直接INSERT-ABSENT(1)，又會破壞哪個不變量？給出安全公開介面及時間界。

參考資料與署名

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing，個人手寫資料結構筆記：佇列、搜尋樹、紅黑樹、多路搜尋樹與堆積。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study